

Progetto di Scalable and Cloud Programming

Federica Santisi

N° matricola 0001186853

Alma Mater Studiorum – Università di Bologna

federica.santisi@studio.unibo.it

Anno Accademico 2025–2026

1 Introduzione

L'obiettivo del progetto è realizzare un'applicazione distribuita in Scala e Apache Spark che, dato un dataset di eventi sismici, identifichi la coppia di località in cui i terremoti co-occorrono (nella stessa finestra temporale di un giorno) con la massima frequenza, elencando in ordine crescente le date in cui tale co-occorrenza si è verificata. L'analisi è eseguita sul dataset completo tramite cluster Apache Spark gestiti su Google Cloud Dataproc, con macchine di tipo `n2-standard-4` come richiesto dalle specifiche del progetto.

L'elaborazione segue il paradigma map-reduce e sfrutta le RDD per controllare esplicitamente le fasi di trasformazione e aggregazione, con particolare attenzione alla riduzione del volume di dati trasferiti durante le fasi di shuffle, che rappresentano il principale collo di bottiglia in questo tipo di elaborazione distribuita.

2 Approccio implementato

2.1 Pre-processing dei dati

Il pre-processing dei dati grezzi consiste nelle seguenti fasi:

- arrotondamento delle coordinate (latitudine e longitudine) alla prima cifra decimale tramite `Math.round(value * 10) / 10.0`, che approssima all'intero più vicino come richiesto dalla traccia, così da raggruppare gli eventi per area geografica anziché per posizione esatta;
- estrazione della sola data dall'istante temporale, in modo che la finestra di co-occorrenza corrisponda a un'intera giornata;
- deduplicazione delle coppie (cella geografica, data), necessaria per evitare che più eventi ricadenti nella stessa cella perchè avvenuti nello stesso giorno, vengano conteggiati come celle distinte nel conteggio delle co-occorrenze;
- gestione robusta degli errori di parsing: le righe con formato non valido (coordinate o date malformate) vengono scartate tramite `Option` e `flatMap`, evitando che una singola riga corrotta faccia fallire l'intero job sul dataset completo.

2.2 Rappresentazione compatta dei dati

Per ridurre l'occupazione di memoria e velocizzare le fasi di shuffle, ogni cella geografica è codificata come un singolo valore `Long` (anziché una tupla `(Double, Double)`), combinando le coordinate arrotondate tramite un offset e un fattore moltiplicativo. Questa scelta è motivata dal fatto che l'hashing e il confronto di un tipo primitivo `Long` sono più leggeri, durante lo shuffle, rispetto al confronto di una tupla, un aspetto che diventa significativo quando il numero di coppie generate cresce in modo combinatorio con il numero di celle attive per giorno.

2.3 Pipeline di elaborazione

La pipeline attuale è organizzata nelle seguenti fasi:

1. **Lettura e ripartizionamento:** il CSV viene letto e ripartizionato in un numero di partizioni configurabile da riga di comando, per poter testare diverse configurazioni senza dover ricompilare il jar;
2. **Aggregazione per data con deduplicazione locale:** le celle vengono aggregate per data tramite `aggregateByKey` usando un `HashSet[Long]` come accumulatore. La deduplicazione avviene così come effetto collaterale dell'aggregazione stessa, senza necessità di uno shuffle stage dedicato. Le date con una sola cella vengono scartate, poiché non possono generare coppie. L'RDD risultante viene mantenuto in cache poiché riutilizzato in due fasi successive;
3. **Generazione delle coppie e conteggio leggero:** per ciascuna data, le celle vengono ordinate e le coppie generate in ordine canonico, evitando confronti aggiuntivi per normalizzarle. Le occorrenze vengono poi

sommate con `reduceByKey(_+_)`, trasferendo nello shuffle solamente coppie di identificatori e un contatore intero, anziché l'elenco completo delle date;

4. **Selezione della coppia vincente:** ottenuta tramite `reduce` distribuito sui conteggi, senza raccogliere l'intero RDD sul driver;
5. **Recupero mirato delle date:** solo a questo punto l'RDD già aggregato e mantenuto in cache al passo 2 viene filtrato per estrarre le sole date in cui compaiono entrambe le celle della coppia vincente, evitando di dover ricostruire da capo l'aggregazione, che rappresenta la fase più costosa dell'intera pipeline. Le date ottenute vengono infine ordinate in modo crescente prima di essere scritte in output, come richiesto dal formato dei risultati.



Figura 1: Diagramma a blocchi della pipeline di elaborazione.

Questa separazione tra una fase di conteggio "leggera" (che trasporta nello shuffle solo un contatore per coppia) e una fase di recupero mirato delle date (eseguita una sola volta, solo per la coppia vincente) è la principale scelta progettuale della pipeline: riduce significativamente il volume di dati trasferiti nello shuffle rispetto a un approccio che porti con sé l'intera lista di date per ogni coppia fin dall'inizio dell'elaborazione.

2.4 Evoluzione del codice e confronto sperimentale

Il codice è stato sviluppato attraverso diverse iterazioni successive, ciascuna motivata da un'ottimizzazione specifica rispetto alla precedente. Le versioni precedenti sono documentate nella cartella `history/` del repository (non fanno parte della build del progetto consegnato) e sono state anche eseguite sul dataset completo, a parità di cluster (3 worker), per quantificare l'impatto delle ottimizzazioni introdotte:

- **Versione 1:** basata su `groupByKey` per raggruppare le celle per data, chiavi (`Double`, `Double`), deduplicazione tramite `.distinct()` separato, riduzione finale con `reduce` semplice;
- **Versione 2:** introduce `aggregateByKey` con deduplicazione locale tramite `Set` come accumulatore, `cache()` sull'RDD normalizzato e `treeReduce` per la riduzione finale, mantenendo comunque le chiavi (`Double`, `Double`);
- **Versione attuale:** introduce la codifica compatta delle celle come `Long`, la deduplicazione integrata nell'`aggregateByKey` per data, il numero di partizioni parametrico da riga di comando, e la separazione tra una fase di conteggio leggero e il recupero mirato delle sole date della coppia vincente.

Versione	Worker	Partizioni	Tempo (s)
v1	3	default	2687,8
v2	3	default	3005,8
Attuale	3	16	300,8

Tabella 1: Confronto sperimentale tra le versioni del codice, stessa configurazione di cluster su dataset completo.

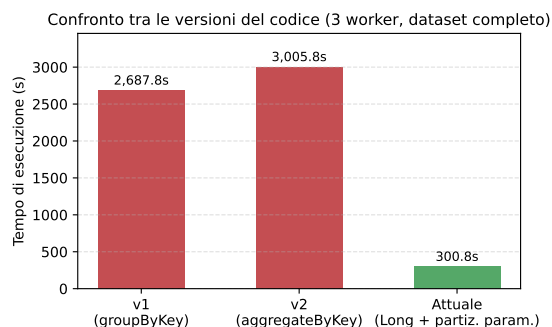


Figura 2: Confronto visivo dei tempi di esecuzione tra le versioni del codice.

Il confronto mostra che sia v1 sia v2 risultano circa 9-10 volte più lente della versione attuale. Un aspetto degno di nota è che v2 non risulta più veloce di v1, nonostante introduca ottimizzazioni algoritmiche aggiuntive (`aggregateByKey`, `cache()`, `treeReduce`): entrambe le versioni precedenti, a differenza di quella attuale, non specificano esplicitamente il numero di partizioni (da cui la dicitura "default" in Tabella 1) e si affidano al parallelismo determinato da Spark in base agli split del file CSV in input. È plausibile che tale parallelismo non sia ottimale per il cluster utilizzato, e che il `cache()` introdotto in v2 aggiunga overhead di materializzazione senza portare benefici, poiché l’RDD su cui è applicato viene utilizzato una sola volta prima dell’aggregazione. Il controllo esplicito sul numero di partizioni è un fattore determinante per le prestazioni, potenzialmente più impattante delle sole ottimizzazioni algoritmiche locali se non accompagnato da un partizionamento adeguato al cluster.

3 Analisi

3.1 Setup sperimentale

Gli esperimenti sono stati eseguiti sul dataset completo, su cluster Google Cloud Dataproc composti da macchine `n2-standard-4` (4 vCPU, 16 GB RAM ciascuna), sia per il nodo master sia per i nodi worker, come richiesto dalle specifiche del progetto. Sono state testate tre dimensioni di cluster (2, 3 e 4 worker) e, per ciascuna, sette valori del numero di partizioni Spark: 4, 8, 16, 32, 64, 128, 256. A questi si aggiungono due run mirati con 12 partizioni.

3.2 Procedura di benchmarking

Per ciascuna combinazione di numero di worker e numero di partizioni, i benchmark sono stati eseguiti manualmente da riga di comando tramite Google Cloud SDK, seguendo per ogni configurazione la stessa sequenza fissa di comandi. Di seguito i comandi utilizzati, con le componenti parametriche variate tra un’esecuzione e l’altra evidenziate in grassetto:

1. **Creazione del cluster** (varia il numero di worker):

```
gcloud dataproc clusters create earthquake-cluster -region=europe-west1 -num-workers <N_WORKER> -master-boot-disk-size 240 -worker-boot-disk-size 240 -master-machine-type=n2-standard-4 -worker-machine-type=n2-standard-4
```

2. **Lancio del job** (varia il numero di partizioni, passato come parametro applicativo, e il percorso di output):

```
gcloud dataproc jobs submit spark -cluster=earthquake-cluster -region=europe-west1 -jar=gs://BUCKET/jars/earthquake-assembly.jar - gs://BUCKET/data/dataset-earthquakes-full.csv gs://BUCKET/output/result <N_PARTITIONS>
```

3. **Recupero del tempo di esecuzione** dai log del job (varia l’identificativo del job):

```
gcloud dataproc jobs wait <JOB_ID> -region=europe-west1 | findstr "Tempo di esecuzione"
```

4. **Eliminazione del cluster**, eseguita al termine di ciascuna serie di test per contenere il consumo delle risorse cloud:

```
gcloud dataproc clusters delete earthquake-cluster -region=europe-west1
```

I comandi 2 e 3 sono stati ripetuti per ciascuno dei sette valori di partizioni testati mantenendo lo stesso cluster attivo, per limitare i tempi complessivi di creazione ed eliminazione; i comandi 1 e 4 sono stati eseguiti una sola volta per ciascuna dimensione di cluster.

3.3 Risultati grezzi

Worker	Partizioni						
	4	8	16	32	64	128	256
2	800,2	512,6	388,2	375,5	340,2	308,6	287,4
3	776,7	478,3	300,8	275,0	250,6	231,2	219,7
4	762,4	528,0	234,0	228,5	199,9	183,6	177,7

Tabella 2: Tempi di esecuzione (s) su cluster `n2-standard-4`, dataset completo.

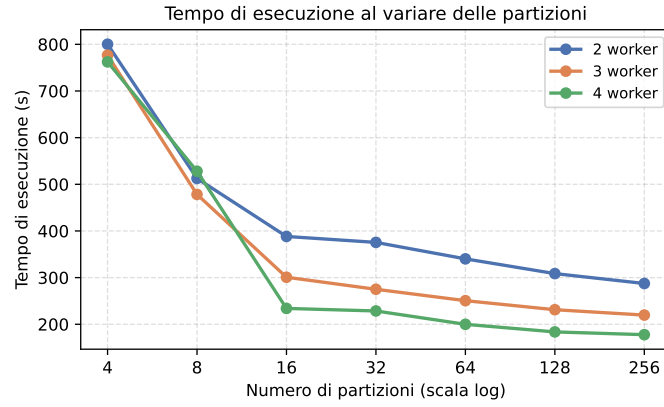


Figura 3: Andamento del tempo di esecuzione al variare delle partizioni (scala log).

3.4 Impatto del numero di partizioni

Ogni nodo worker `n2-standard-4` dispone di 4 vCPU; un cluster a 2, 3 e 4 worker mette quindi a disposizione rispettivamente 8, 12 e 16 core. Con 4 partizioni i tempi sono quasi indipendenti dal numero di worker (800,2s–762,4s, variazione inferiore al 5%). A 8 partizioni sul cluster a 3 worker, una prima misurazione (533,7s) risultava superiore a quella a 2 worker (512,6s), un esito anomalo rispetto alle attese; il test è stato ripetuto ottenendo 478,3s, valore riportato in tabella. Anche corretto questo dato, il tempo a 8 partizioni su 4 worker (528,0s) resta comunque superiore sia a 2 sia a 3 worker. La causa di fondo, comune a 4 e 8 partizioni, è che il numero di task generati è insufficiente a saturare i core disponibili anche sul cluster più piccolo: aumentare i worker non aiuta se il collo di bottiglia è il numero di task concorrenti, non le risorse. Solo da 16 partizioni in poi, quando i task superano i core anche su 3 e 4 worker, il parallelismo aggiuntivo produce un beneficio misurabile (388,2s, 300,8s, 234,0s). L'appiattimento oltre questa soglia era atteso, ma è stato comunque testato un intervallo di partizioni più ampio (fino a 256) per quantificarne l'entità anziché limitarsi a confermarlo qualitativamente.

3.4.1 Verifica mirata della soglia partizioni–core

Per verificare direttamente questa ipotesi sono stati eseguiti due run aggiuntivi con lo stesso numero di partizioni (12, pari esattamente al numero di core disponibili sul cluster a 3 worker) su due configurazioni di cluster diverse:

Worker	Core disponibili	Partizioni	Tempo (s)
3	12	12	295,0
4	16	12	275,6

Tabella 3: Run mirati per isolare l'effetto soglia partizioni = core disponibili.

Il confronto conferma l'ipotesi su due fronti. Da un lato, a parità di worker (3), il tempo a 12 partizioni (295,0s) è coerente con quello a 16 partizioni (300,8s): con 12 partizioni si saturano esattamente i 12 core disponibili, per cui aumentare ulteriormente le partizioni oltre le esigenze del cluster non produce benefici proporzionali all'aumento di partizioni, per quanto il tempo di esecuzione continui a diminuire leggermente. Dall'altro lato, a parità di partizioni (12), il cluster a 4 worker (16 core) è comunque più veloce di quello a 3 worker (275,6s contro 295,0s) grazie ai core aggiuntivi disponibili, ma resta più lento del proprio valore a 16 partizioni (234,0s): con 12 partizioni, 4 dei 16 core del cluster a 4 worker restano inevitabilmente inattivi, poiché il numero di task è inferiore al numero di core. Questo dimostra che l'effetto osservato dipende dal rapporto tra partizioni e core disponibili sul cluster specifico, e non dal numero assoluto di partizioni.

3.5 Strong scaling: speedup ed efficienza

Speedup ed efficienza sono calcolati usando il miglior tempo ottenuto per ciascuna configurazione di cluster (256 partizioni in tutti e tre i casi), rispetto al baseline a 2 worker.

Worker	Best time (s)	Speedup	Efficienza
2	287,4	1,00	1,000
3	219,7	1,31	0,872
4	177,7	1,62	0,809

Tabella 4: Speedup ed efficienza rispetto al baseline a 2 worker (miglior tempo per configurazione).

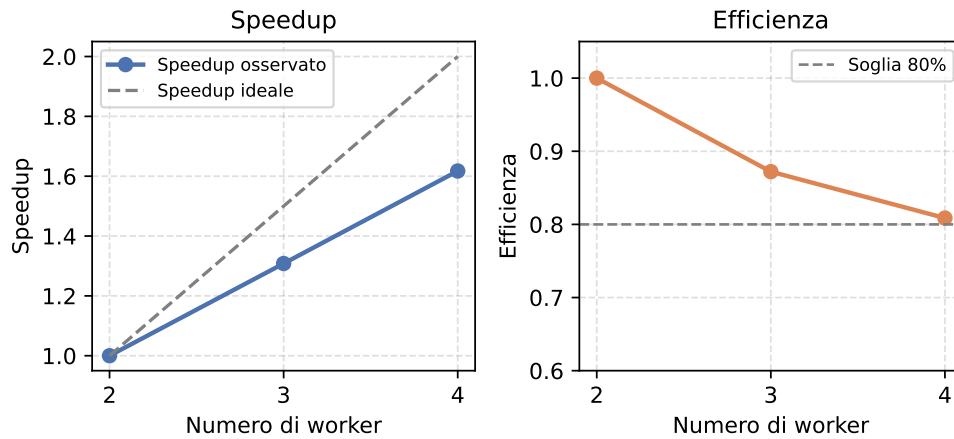


Figura 4: Speedup ed efficienza rispetto al baseline a 2 worker.

L'efficienza decresce in modo contenuto e progressivo all'aumentare del numero di worker (dal 100% al 87,2% fino all'80,9%), indicando una buona scalabilità della pipeline anche al crescere della dimensione del cluster, senza segnali di plateau marcato nell'intervallo testato.

3.6 Legge di Amdahl

A partire dallo speedup massimo osservato ($S = 1,62$ passando da 2 a 4 worker, quindi $p = 2$), la legge di Amdahl permette di stimare la frazione seriale f del workload tramite la relazione $S = 1/(f + (1 - f)/p)$:

$$f \approx 0,24 \Rightarrow \text{frazione parallelizzabile} \approx 76\%$$

Questo risultato indica una pipeline complessivamente ben parallelizzata, con una componente seriale e di overhead (shuffle, sincronizzazione tra stage, scheduling, tempo di setup del job e del cluster) stimabile attorno al 24%. Tale quota non è interamente eliminabile per costruzione: fasi come la selezione della coppia vincente e l'avvio del job hanno una componente intrinsecamente non parallelizzabile, indipendentemente dal numero di worker disponibili.

4 Conclusioni

L'approccio basato su `aggregateByKey` con `HashSet` per la deduplicazione locale, codifica compatta delle celle come `Long` e separazione tra conteggio leggero e recupero mirato delle date, permette di eseguire l'analisi sul dataset completo in tempi dell'ordine dei minuti anche su cluster di dimensioni contenute. Il numero di partizioni ha un impatto rilevante sulle prestazioni: partizionare in modo insufficiente rispetto ai core disponibili sul cluster lascia risorse inattive (le configurazioni a 4 partizioni sono fino a circa 4-4,5 volte più lente di quelle a 256), effetto confermato direttamente dai run mirati sulla soglia partizioni-core. La scalabilità al crescere del numero di worker risulta buona, con un'efficienza che si mantiene sopra l'80% anche a 4 worker.

Il numero di co-occorrenze trovate è **10.032** (giorni distinti in cui entrambe le celle della coppia hanno registrato un evento sismico) con la coppia $((38.8, -122.8), (38.8, -122.7))$.

```

Output      A capo automatico: Off
26/07/26 21:35:22 INFO GoogleCloudStorageImpl: Ignoring exception of type GoogleJsonResponseException; verified objec
26/07/26 21:35:22 INFO GoogleHadoopOutputStream: hflush(): No-op due to rate limit (RateLimiter[stableRate=0.2qps]):
[INFO] numPartitions=256, defaultParallelism=2
[INFO] Tempo di esecuzione: 177714ms
((38.8,-122.8), (38.8,-122.7))
1990-01-05
1990-01-06
1990-01-07
1990-01-09
1990-01-11
1990-01-13

```

Figura 5: Output del job con la configurazione a prestazioni migliori (4 worker, 256 partizioni): tempo di esecuzione, coppia vincente e date di co-occorrenza (elenco troncato per motivi di spazio).

Codice sorgente

Il codice sorgente del progetto, comprensivo di README con le istruzioni per l'esecuzione su Google Cloud Dataproc, è disponibile al seguente repository pubblico: <https://github.com/santisifederica/scala-project.git>